

¿Qué es un ORM y para qué sirve?

Explicación sencilla, diferencias con SQL y ejemplos en Python

Cuando trabajamos con programas en Python que necesitan guardar información en una base de datos, podemos hacerlo de dos formas: escribiendo consultas en código SQL directamente o usando un ORM. A continuación explico de forma práctica qué es, cómo funciona y cuándo conviene usarlo.

1. Concepto principal

ORM significa Object-Relational Mapping (Mapeo Objeto-Relacional). En la práctica, es una herramienta que funciona como puente o traductor entre tu programa y la base de datos.

En lugar de interactuar con tablas, filas y columnas escribiendo texto en SQL, un ORM te permite tratar los datos como si fueran objetos normales de tu lenguaje de programación:

- **Las tablas** pasan a ser clases en tu código (por ejemplo, la clase Usuario).
- **Las filas** son las instancias u objetos de esa clase.
- **Las columnas** son los atributos del objeto (por ejemplo, usuario.nombre o usuario.email).

2. Comparativa: SQL Nativo vs. ORM

Aspecto	SQL Nativo	Uso de ORM (ej. SQLAlchemy)
Forma de trabajo	Se escriben consultas manuales (SELECT, INSERT).	Se usan métodos y funciones nativas de Python.
Enfoque	Orientado a tablas y relaciones de la BD.	Orientado a objetos y clases.
Cambios de base de datos	Requiere revisar y adaptar consultas si cambia el motor.	El ORM adapta las consultas automáticamente.
Seguridad	Exige validar manualmente los datos para evitar Inyección SQL.	Limpia y valida los parámetros de entrada de forma automática.

3. Ejemplos comparativos

Consulta 1: Filtrar registros (Usuarios mayores de 18 años)

Escribiendo SQL directo:
`SELECT * FROM usuarios WHERE edad > 18;`

Usando un ORM en Python:

```
usuarios_mayores = Usuario.objects.filter(edad__gt=18)
```

Consulta 2: Insertar un nuevo registro

Escribiendo SQL directo:

```
INSERT INTO usuarios (nombre, edad) VALUES ('Carlos', 25);
```

Usando un ORM en Python:

```
nuevo_usuario = Usuario(nombre='Carlos', edad=25)  
nuevo_usuario.save()
```

4. Conclusión: ¿Cuál utilizar?

- **Conviene usar ORM** en la gran mayoría de los desarrollos web o de software. Agiliza la escritura de código, facilita el mantenimiento del sistema y previene fallos comunes de seguridad.
- **Conviene usar SQL Nativo** únicamente cuando se realizan reportes muy complejos, consultas masivas o cuando se requiere optimizar el rendimiento al máximo nivel.